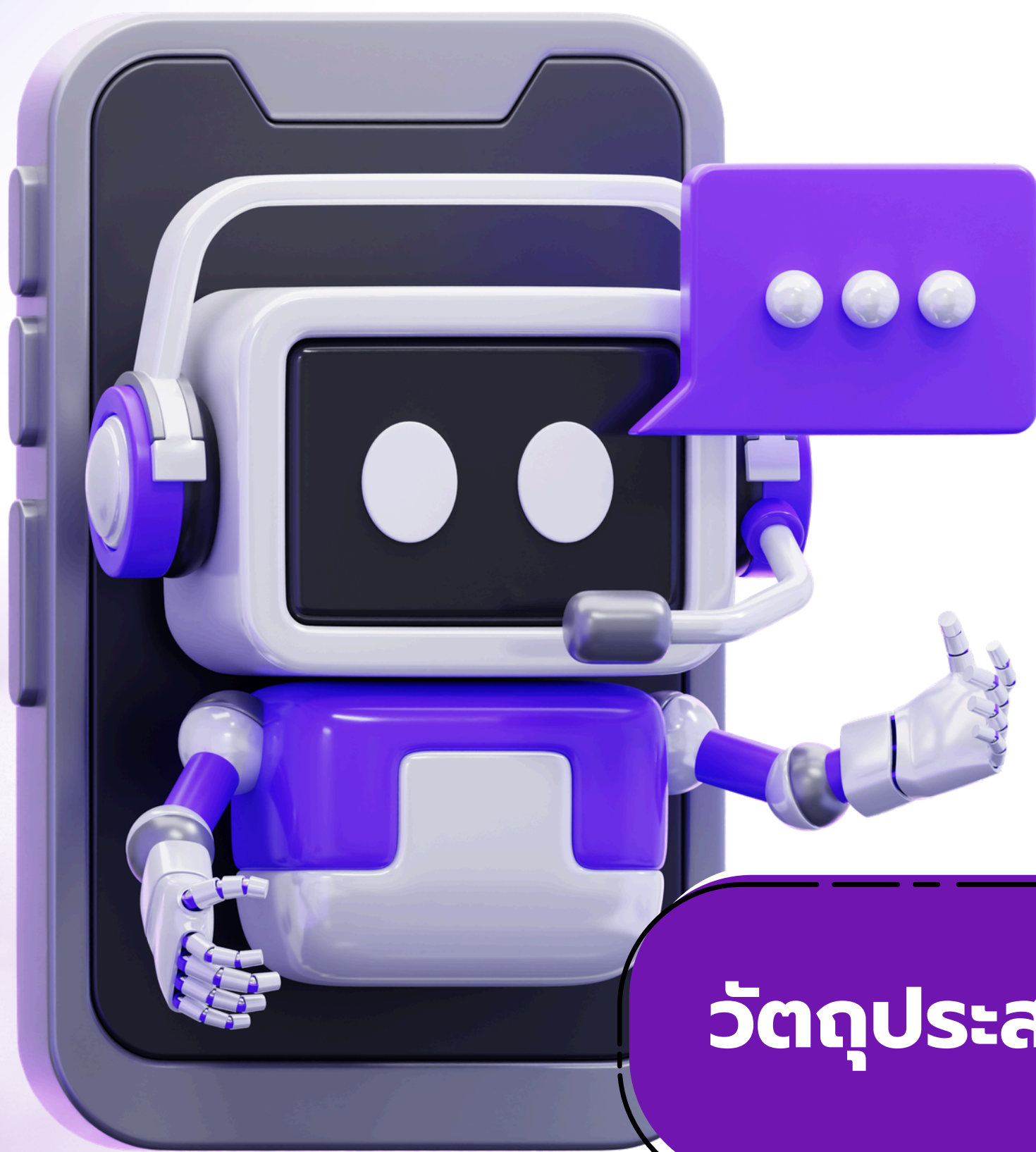


หน่วยที่ 2

ภาษาในการเรียนโปรแกรมเชิงวัตถุเบื้องต้น



วัตถุประสงค์เชิงพฤติกรรม

1. อธิบายความเป็นมา ความสำคัญ และคุณลักษณะเด่นของภาษา C#
2. บอกองค์ประกอบโครงสร้างโปรแกรมและสภาพแวดล้อมการพัฒนา
3. จำแนกชนิดข้อมูลและตัวแปรในภาษา C#
4. เขียนโปรแกรม C# เบื้องต้นเพื่อประกาศคลาส สร้างวัตถุ และเรียกใช้เมธอด
5. อธิบายและยกตัวอย่างการนำ OOP (Encapsulation, Inheritance, Polymorphism, Abstraction) ไปใช้ในภาษา C#



หน่วยที่ 2 ภาษาในการเรียนโปรแกรมเชิงวัตถุเบื้องต้น

วัตถุประสงค์เชิงพฤติกรรม

1. อธิบายความเป็นมา ความสำคัญ และคุณลักษณะเด่นของภาษา C#
2. บอกองค์ประกอบโครงสร้างโปรแกรมและสภาพแวดล้อมการพัฒนา
3. จำแนกชนิดข้อมูลและตัวแปรในภาษา C#
4. เขียนโปรแกรม C# เบื้องต้นเพื่อประกาศคลาส สร้างวัตถุ และเรียกใช้เมธอด
5. อธิบายและยกตัวอย่างการนำ OOP (Encapsulation, Inheritance, Polymorphism, Abstraction) ไปใช้ในภาษา C#

2.1 ความเป็นมาและความสำคัญของภาษา C#

2.1.1 ประวัติความเป็นมา

ภาษา C# (อ่านว่า "ซีชาร์ป") เป็นภาษาโปรแกรมเชิงวัตถุที่พัฒนาโดยบริษัทไมโครซอฟท์ (Microsoft) เริ่มพัฒนาในปี ค.ศ. 1999 และเปิดตัวอย่างเป็นทางการในปี ค.ศ. 2000 พร้อมกับแพลตฟอร์ม .NET Framework เวอร์ชันแรก

บุคคลสำคัญที่อยู่เบื้องหลังการออกแบบภาษานี้คือ **Anders Hejlsberg** วิศวกรซอฟต์แวร์ชาวเดนมาร์ก ผู้มีประวัติผลงานโดดเด่นมาก่อนหน้านี้ เขาเคยเป็นหัวหน้าทีมพัฒนา **Turbo Pascal** และ **Delphi** ที่บริษัท Borland ซึ่งเป็นเครื่องมือพัฒนาโปรแกรมที่โด่งดังในยุค 1980-1990 ก่อนจะย้ายมาร่วมงานกับไมโครซอฟท์และรับหน้าที่เป็นสถาปนิกหลัก (Chief Architect) ในการออกแบบภาษา C#

แนวคิดเบื้องหลังการออกแบบ: ทีมออกแบบต้องการสร้างภาษาที่ผสมผสานจุดเด่นของภาษาโปรแกรมต่าง ๆ เข้าด้วยกัน โดยนำไวยากรณ์ที่คุ้นเคยจากภาษา C++ และ Java มาปรับปรุงให้เขียนง่ายขึ้น ปลอดภัยขึ้น และรองรับแนวคิดเชิงวัตถุอย่างสมบูรณ์ ตั้งแต่การออกแบบภาษา (ต่างจากภาษา C++ ที่เพิ่มความสามารถเชิงวัตถุเข้าไปภายหลัง)



ภาพที่ 1 ภาษา C#

ที่มา <https://www.educative.io/blog/is-c-sharp-and-net-still-relevant>

2.1.2 พัฒนาการของภาษา C# ตลอดหลายปีที่ผ่านมา

ภาษา C# มีการพัฒนาเวอร์ชันควบคู่ไปกับแพลตฟอร์ม .NET อย่างต่อเนื่องทุกปี โดยแต่ละเวอร์ชันจะเพิ่มความสามารถใหม่เพื่อให้เขียนโค้ดได้กระชับ ปลอดภัย และมีประสิทธิภาพยิ่งขึ้น

ช่วงเวลา	เวอร์ชันภาษา/แพลตฟอร์ม	ความสำคัญ
ค.ศ. 2000–2002	C# 1.0 / .NET Framework 1.0	เปิดตัวภาษา C# อย่างเป็นทางการ
ค.ศ. 2005	C# 2.0	เพิ่ม Generics, Nullable Types
ค.ศ. 2007	C# 3.0	เพิ่ม LINQ (Language Integrated Query)
ค.ศ. 2014	เปิดซอร์สโค้ด .NET	ไมโครซอฟท์เปิดโครงการ .NET Foundation ให้เป็นโอเพนซอร์ซ
ค.ศ. 2016–2019	.NET Core / C# 7–8	รองรับการทำงานข้ามแพลตฟอร์ม (Windows, macOS, Linux)
ค.ศ. 2021	.NET 6 (LTS)	รวม .NET Framework, .NET Core และ Xamarin เข้าเป็นแพลตฟอร์มเดียว
พ.ศ. 2568 (ค.ศ. 2025)	C# 14 / .NET 10 (LTS)	เวอร์ชันเสถียรล่าสุดที่ใช้งานจริงในอุตสาหกรรม

จุดสังเกตสำคัญ: เวอร์ชันของภาษา C# ผูกติดกับเวอร์ชันของ .NET เสมอ หากต้องการใช้ความสามารถใหม่ล่าสุดของภาษา จำเป็นต้องอัปเดตแพลตฟอร์ม .NET ให้เป็นเวอร์ชันที่รองรับด้วย

2.1.3 ความสำคัญของภาษา C# ในอุตสาหกรรมซอฟต์แวร์

 งานพัฒนาโปรแกรมองค์กร (Enterprise Application)

บริษัทและหน่วยงานจำนวนมากใช้ C# พัฒนาระบบสารสนเทศ ระบบบัญชี และระบบจัดการฐานข้อมูลบนระบบปฏิบัติการ Windows เนื่องจากมีความเสถียร ปลอดภัย และรองรับการทำงานร่วมกับผลิตภัณฑ์อื่นของไมโครซอฟท์ได้ดี

 งานพัฒนาเว็บแอปพลิเคชัน

ผ่านเฟรมเวิร์ก **ASP.NET Core** ซึ่งรองรับการสร้างเว็บไซต์และ Web API ที่มีประสิทธิภาพสูง เป็นที่นิยมในการพัฒนาระบบ Backend ขนาดใหญ่

 งานพัฒนาเกม

ผ่านโปรแกรม **Unity** ซึ่งเป็นเกมเอนจิน (Game Engine) ที่ได้รับความนิยมสูงที่สุดแห่งหนึ่งของโลก ใช้ C# เป็นภาษาหลักในการเขียนสคริปต์ควบคุมพฤติกรรมของตัวละครและระบบเกม นักพัฒนาเกมอิสระ (Indie Developer) และสตูดิโอขนาดใหญ่จำนวนมากเลือกใช้ Unity ร่วมกับ C# ในการสร้างเกมทั้งบนมือถือ พีซี และคอนโซล



งานพัฒนาแอปพลิเคชันข้ามแพลตฟอร์ม

ผ่าน .NET MAUI (Multi-platform App UI) ที่สามารถเขียนโค้ดครั้งเดียวแล้วนำไปใช้งานได้ทั้งบน Windows, macOS, Android และ iOS ช่วยลดเวลาและต้นทุนในการพัฒนาแอปพลิเคชันสำหรับหลายแพลตฟอร์ม

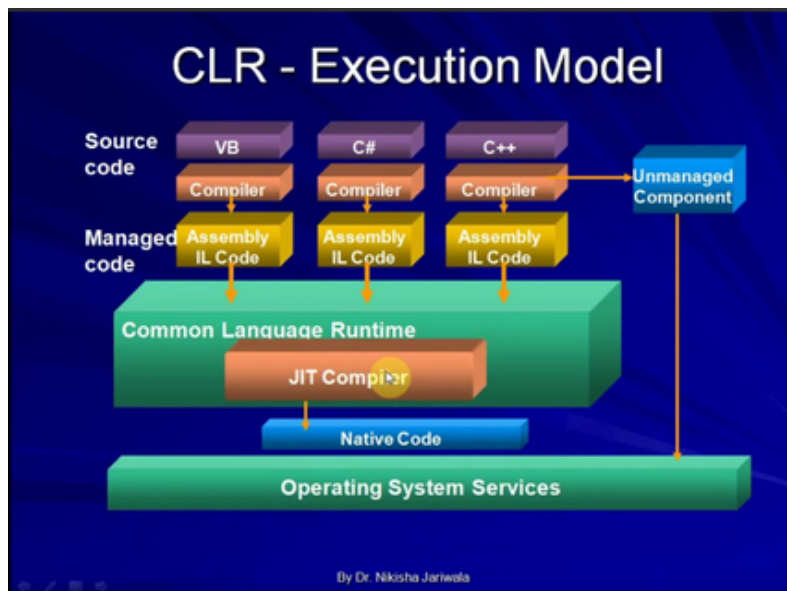


ตลาดแรงงาน

C# เป็นหนึ่งในภาษาที่มีตำแหน่งงานรองรับสูงต่อเนื่องหลายปี เหมาะสำหรับผู้เรียนสายอาชีพที่ต้องการทำงานด้านพัฒนาซอฟต์แวร์ในองค์กรขนาดกลางถึงขนาดใหญ่

2.1.4 คุณลักษณะเด่นของภาษา C#

การทำความเข้าใจคุณลักษณะของภาษา C# จะช่วยให้ผู้เรียนมองเห็นภาพว่าเหตุใดภาษานี้จึงถูกเลือกใช้ในงานพัฒนาซอฟต์แวร์ระดับองค์กร



ภาพที่ 2 คุณลักษณะของภาษา C#

ที่มา <https://www.youtube.com/watch?v=MwJTGsnIWXs>

1. เป็นภาษาเชิงวัตถุแท้ (Pure Object-Oriented Language)

ทุกองค์ประกอบในโปรแกรม C# ไม่ว่าจะเป็นตัวแปร ฟังก์ชัน หรือแม้แต่ชนิดข้อมูลพื้นฐาน ล้วนอ้างอิงอยู่บนแนวคิดของคลาสทั้งสิ้น ภาษา C# จึงรองรับคุณสมบัติ OOP ทั้ง 4 ประการ (Encapsulation, Inheritance, Polymorphism, Abstraction) ได้อย่างสมบูรณ์และเป็นธรรมชาติ

2. Type-Safe (การกำหนดชนิดข้อมูลอย่างเข้มงวด)

ตัวแปรทุกตัวในภาษา C# ต้องมีการประกาศชนิดข้อมูลอย่างชัดเจนก่อนใช้งาน และตัวแปลภาษา (Compiler) จะตรวจสอบความถูกต้องของชนิดข้อมูลตั้งแต่ขั้นตอนคอมไพล์ ก่อนที่โปรแกรมจะถูกรัน ซึ่งช่วยลดข้อผิดพลาดขณะโปรแกรมทำงานจริงได้เป็นอย่างมาก

3. ทำงานบน Common Language Runtime (CLR)

โค้ดภาษา C# จะไม่ถูกแปลงเป็นภาษาเครื่องโดยตรง แต่จะถูกแปลงเป็นภาษากลางที่เรียกว่า **Intermediate Language (IL)** ก่อน แล้วจึงถูกแปลงเป็นภาษาเครื่องอีกครั้งขณะรันโปรแกรมโดย CLR ทำให้โปรแกรม C# สามารถทำงานได้บนระบบปฏิบัติการที่หลากหลาย ตราบใดที่เครื่องนั้นมี .NET Runtime ติดตั้งอยู่

4. มีการจัดการหน่วยความจำอัตโนมัติ (Automatic Memory Management)

ภาษา C# มีกลไกที่เรียกว่า **Garbage Collector (GC)** ทำหน้าที่ตรวจสอบและคืนพื้นที่หน่วยความจำของวัตถุที่ไม่มีการอ้างอิงใช้งานแล้วโดยอัตโนมัติ ผู้เขียนโปรแกรมจึงไม่จำเป็นต้องเขียนคำสั่งคืนหน่วยความจำเองเหมือนภาษา C หรือ C++

5. ไวยากรณ์ใกล้เคียงตระกูลภาษา C

โครงสร้างคำสั่ง เช่น การใช้เครื่องหมายปีกกา { } กำหนดขอบเขต การใช้เครื่องหมายเซมิโคลอน ; ปิดท้ายคำสั่ง มีความคล้ายคลึงกับภาษา C, C++ และ Java เป็นอย่างมาก ทำให้ผู้เรียนที่มีพื้นฐานภาษาตระกูลนี้สามารถปรับตัวเรียนรู้ภาษา C# ได้อย่างรวดเร็ว

6. มีไลบรารีมาตรฐานจำนวนมาก (.NET Class Library)

.NET มีชุดคำสั่งสำเร็จรูปให้เรียกใช้งานได้ทันทีโดยไม่ต้องเขียนโค้ดใหม่ทั้งหมด ครอบคลุมงานหลากหลายประเภท เช่น การอ่าน-เขียนไฟล์ การเชื่อมต่อฐานข้อมูล การจัดการเครือข่าย และการประมวลผลข้อความ

7. โอเพนซอร์ซและใช้งานข้ามแพลตฟอร์ม (Open Source & Cross-Platform)

ตั้งแต่ปี ค.ศ. 2014 เป็นต้นมา ไมโครซอฟท์เปิดซอร์สโค้ดของภาษา C# และแพลตฟอร์ม .NET ผ่านโครงการ **.NET Foundation** ทำให้นักพัฒนาทั่วโลกสามารถศึกษา ใช้งาน และร่วมพัฒนาได้โดยไม่มีค่าใช้จ่าย อีกทั้งยังสามารถติดตั้งและใช้งานได้บนระบบปฏิบัติการ Windows, macOS และ Linux

2.2 องค์ประกอบโครงสร้างโปรแกรมและสภาพแวดล้อมการพัฒนา

2.2.1 .NET SDK (Software Development Kit)

.NET SDK คือชุดเครื่องมือหลักที่จำเป็นสำหรับการพัฒนาโปรแกรม C# ทุกโปรแกรม โดยไม่มี .NET SDK ผู้เรียนจะไม่สามารถคอมไพล์หรือรันโปรแกรม C# ได้เลย ภายใน .NET SDK ประกอบด้วยองค์ประกอบย่อยที่สำคัญ 4 ส่วน ดังนี้

องค์ประกอบ	หน้าที่
ตัวแปลภาษา (Compiler)	แปลงซอร์สโค้ด C# ให้เป็นภาษากลาง (Intermediate Language)
ไลบรารีมาตรฐาน (Base Class Library)	ชุดคำสั่งสำเร็จรูปสำหรับใช้งานทั่วไป เช่น การจัดการไฟล์ ข้อความ
รันไทม์ (Runtime)	สภาพแวดล้อมที่จำเป็นต่อการรันโปรแกรมที่คอมไพล์แล้ว
เครื่องมือบรรทัดคำสั่ง (CLI)	คำสั่งจัดการโปรเจกต์ เช่น dotnet new, dotnet build, dotnet run

คำสั่ง CLI ที่ควรรู้เบื้องต้น

คำสั่ง	ความหมาย
dotnet new console	สร้างโปรเจกต์ Console Application ใหม่
dotnet build	คอมไพล์โปรแกรมโดยไม่รัน
dotnet run	คอมไพล์และรันโปรแกรมในขั้นตอนเดียว
dotnet --version	ตรวจสอบเวอร์ชันของ .NET SDK ที่ติดตั้งอยู่

ผู้เรียนสามารถดาวน์โหลด .NET SDK ได้ฟรีจากเว็บไซต์ทางการ <https://dotnet.microsoft.com> โดยรองรับการติดตั้งบนระบบปฏิบัติการ Windows, macOS และ Linux

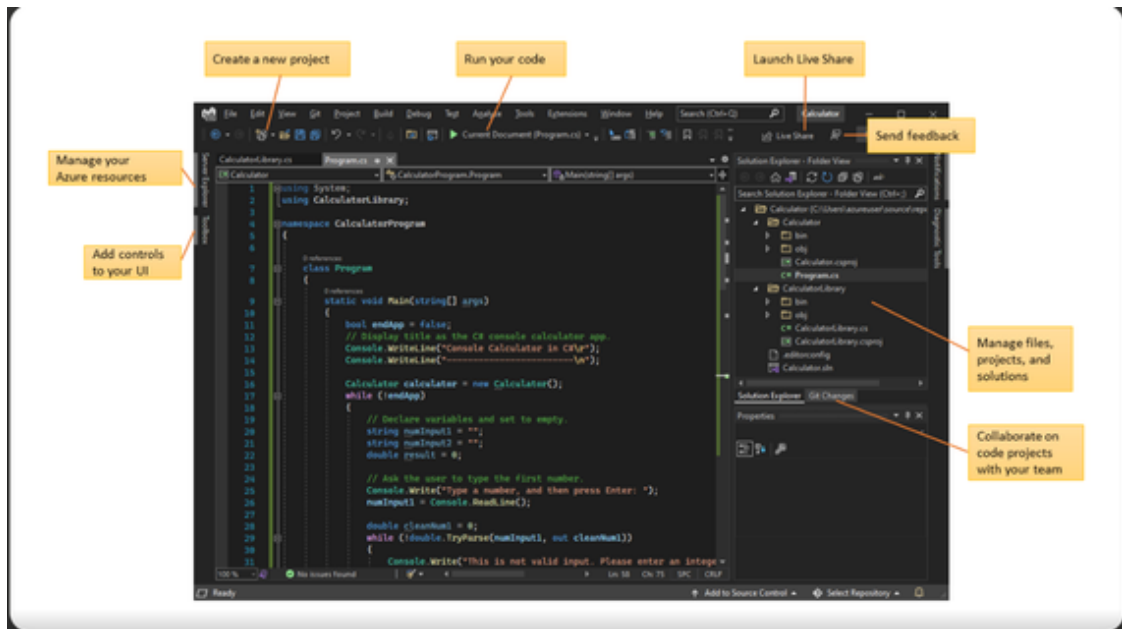
2.2.2 Visual Studio

Visual Studio คือโปรแกรม IDE (Integrated Development Environment) แบบครบวงจรที่ไม่โครซอฟท์พัฒนาขึ้นโดยเฉพาะสำหรับงาน .NET เป็นเครื่องมือที่ผู้เริ่มต้นแนะนำให้ใช้มากที่สุด เนื่องจากมีความสามารถครบถ้วนในตัว ไม่ต้องติดตั้งส่วนขยายเพิ่มเติม คุณสมบัติเด่นได้แก่

คุณสมบัติ	รายละเอียด
IntelliSense	ระบบช่วยเขียนโค้ดอัจฉริยะที่แนะนำคำสั่งและตรวจสอบไวยากรณ์แบบเรียลไทม์ขณะพิมพ์
Debugger	เครื่องมือดีบั๊กที่ช่วยตรวจสอบค่าตัวแปรและลำดับการทำงานของโปรแกรมทีละขั้นตอน (Step-by-step)
Designer	เครื่องมือออกแบบหน้าจอโปรแกรมแบบลากวาง (Drag and Drop) สำหรับงาน Windows Forms หรือ WPF
Solution Explorer	หน้าต่างแสดงโครงสร้างไฟล์และโปรเจกต์ทั้งหมดอย่างเป็นระบบ

รุ่นของ Visual Studio ที่มีให้เลือก

- **Community** — ใช้งานได้ฟรีสำหรับนักเรียน นักศึกษา และผู้พัฒนารายบุคคล (แนะนำสำหรับผู้เรียน ปวช.)
- **Professional** — สำหรับทีมพัฒนาขนาดเล็กถึงกลาง มีค่าใช้จ่ายรายปี
- **Enterprise** — สำหรับองค์กรขนาดใหญ่ มีเครื่องมือทดสอบและวิเคราะห์ขั้นสูง



ภาพที่ 3 หน้าจอ Visual Studio

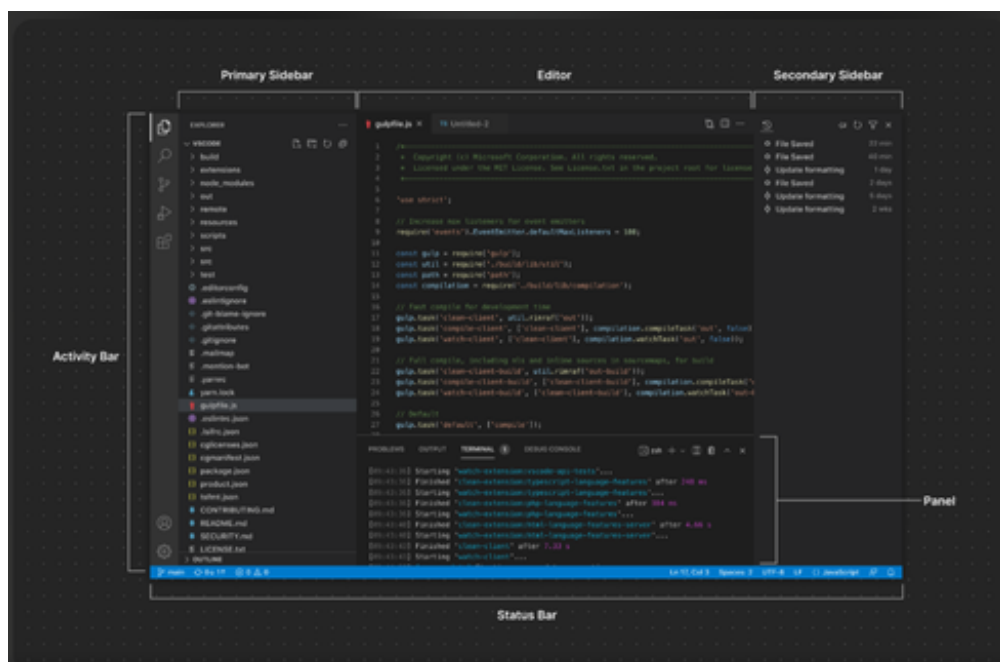
ที่มา <https://devworks.jp/blog/267>

2.2.3 Visual Studio Code

Visual Studio Code (VS Code) คือโปรแกรมแก้ไขโค้ดขนาดเล็ก น้ำหนักเบา พัฒนาโดยไมโครซอฟท์เช่นกัน แต่มีความแตกต่างจาก Visual Studio ตรงที่ **ไม่ใช่ IDE เฉพาะทางสำหรับ .NET โดยตรง** เป็นเพียง Code Editor อเนกประสงค์ที่รองรับได้หลายภาษาโปรแกรม

จุดเด่นของ VS Code

- ใช้งานได้ฟรีบนทุกระบบปฏิบัติการ (Windows, macOS, Linux)
- ใช้ทรัพยากรเครื่องน้อยกว่า Visual Studio มาก เหมาะกับเครื่องคอมพิวเตอร์สเปกไม่สูง
- เปิดโปรแกรมได้รวดเร็ว มีส่วนขยาย (Extension) ให้เลือกติดตั้งเพิ่มเติมได้หลากหลาย
- ต้องติดตั้งส่วนขยายชื่อ "C# Dev Kit" เพิ่มเติมเพื่อให้รองรับการเขียนและดีบั๊กโปรแกรม C#

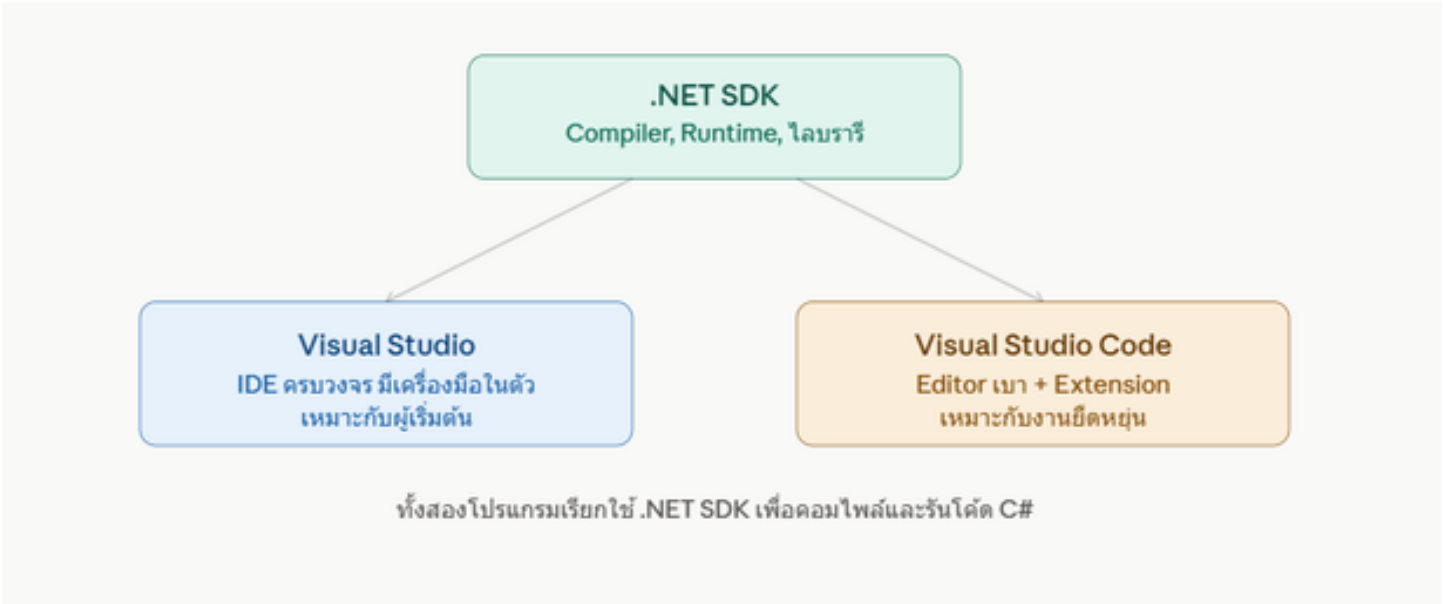


ที่มา <https://qna.wellkeptbra.com/visual-studio-code-interface-visual-studio-code-windows-10/>

2.2.4 ตารางเปรียบเทียบเครื่องมือพัฒนา

หัวข้อเปรียบเทียบ	Visual Studio	Visual Studio Code
ลักษณะ	IDE แบบครบวงจร	Code Editor หน้าหนักเบา
การติดตั้งเพิ่มเติม	มีเครื่องมือ .NET ในตัว	ต้องติดตั้ง Extension เพิ่ม
การใช้ทรัพยากรเครื่อง	สูง	ต่ำ
ความเร็วในการเปิดโปรแกรม	ช้ากว่า	เร็วกว่า
ความเหมาะสม	ผู้เริ่มต้น งานขนาดใหญ่ งานออกแบบหน้าจอ	ผู้มีประสบการณ์ งานหลายภาษา เครื่องสเปกไม่สูง
ค่าใช้จ่าย	ฟรี (รุ่น Community)	ฟรีทั้งหมด

ต่อไปนี้เป็นแผนภาพสรุปความสัมพันธ์ขององค์ประกอบทั้งหมดในสภาพแวดล้อมการพัฒนา C#



ภาพที่ 5 ความสัมพันธ์ขององค์ประกอบ

2.2.5 ขั้นตอนเบื้องต้นในการเริ่มต้นพัฒนาโปรแกรม C#

1. ติดตั้ง .NET SDK จากเว็บไซต์ทางการ (จำเป็นเสมอ ไม่ว่าจะใช้เครื่องมือใด)
2. ติดตั้ง Visual Studio Community หรือ Visual Studio Code พร้อมส่วนขยาย C# Dev Kit
3. สร้างโปรเจกต์ใหม่ประเภท Console Application
4. เขียนโค้ดในไฟล์ที่มีนามสกุล .cs
5. คอมไพล์และรันโปรแกรมเพื่อตรวจสอบผลลัพธ์
6. แก้ไขข้อผิดพลาด (ถ้ามี) โดยอาศัยข้อความแจ้งเตือนจากตัวแปลภาษา

2.3 การจำแนกชนิดข้อมูลและตัวแปรในภาษา C#

2.3.1 หลักการของ Strongly Typed Language

ภาษา C# เป็นภาษาที่มีการกำหนดชนิดข้อมูลอย่างเข้มงวด (Strongly Typed) หมายความว่าตัวแปรทุกตัวต้องประกาศชนิดข้อมูลอย่างชัดเจนก่อนใช้งาน และเมื่อกำหนดชนิดข้อมูลแล้วจะไม่สามารถเปลี่ยนไปเก็บค่าชนิดอื่นได้โดยตรง เช่น หากประกาศตัวแปรเป็น int แล้ว จะไม่สามารถนำข้อความ (string) ไปเก็บในตัวแปรนั้นได้

หลักการนี้ช่วยให้ตัวแปลภาษา (Compiler) ตรวจพบข้อผิดพลาดได้ตั้งแต่ขั้นตอนคอมไพล์ ก่อนที่โปรแกรมจะถูกนำไปใช้งานจริง ซึ่งลดความเสี่ยงของข้อผิดพลาดขณะโปรแกรมทำงาน (Run-time Error) ได้อย่างมาก

2.3.2 ชนิดข้อมูลพื้นฐาน (Primitive Data Types)

ชนิดข้อมูล	คำสั่งประกาศ	ขนาด (บิต)	ตัวอย่างค่า	ความหมาย
จำนวนเต็ม	int	32	int age = 18;	เก็บเลขจำนวนเต็ม ตั้งแต่ -2,147,483,648 ถึง 2,147,483,647
จำนวนเต็มขนาดใหญ่	long	64	long population = 70000000L;	เก็บเลขจำนวนเต็มขนาดใหญ่
ทศนิยม	double	64	double gpa = 3.75;	เก็บเลขทศนิยมความละเอียดสูง นิยมใช้มากที่สุด
ทศนิยมละเอียดสูง	decimal	128	decimal price = 199.50m;	เหมาะกับการเงินที่ต้องการความแม่นยำสูง
ข้อความ	string	-	string name = "สมชาย";	เก็บข้อความ/ตัวอักษรหลายตัวเรียงกัน
ตัวอักษรเดี่ยว	char	16	char grade = 'A';	เก็บตัวอักษรเพียงหนึ่งตัว ต้องใช้เครื่องหมายคำพูดเดี่ยว
ค่าความจริง	bool	-	bool isPassed = true;	เก็บค่าตรรกะได้เพียง true หรือ false

ข้อสังเกต: ชนิดข้อมูลตัวเลขมีหลายแบบเพื่อให้เลือกใช้ตามความเหมาะสมของขนาดข้อมูลและความแม่นยำที่ต้องการ เช่น งานทางการเงินควรใช้ decimal เพราะมีความแม่นยำสูงกว่า double

2.3.3 ค่าคงที่ (Constant)

ค่าคงที่คือค่าที่กำหนดไว้ตั้งแต่ต้นและไม่สามารถเปลี่ยนแปลงได้ในภายหลังตลอดการทำงานของโปรแกรม ใช้คำสั่ง const นำหน้าการประกาศ

```
csharp
const double PI = 3.14159;
const int MaxStudents = 40;
```

หากพยายามเขียนคำสั่งกำหนดค่าใหม่ให้กับตัวแปรที่ประกาศเป็น const ตัวแปลภาษาจะแจ้งข้อผิดพลาดทันทีตั้งแต่ขั้นตอนคอมไพล์

2.3.4 กฎการตั้งชื่อตัวแปรในภาษา C#

- ต้องขึ้นต้นด้วยตัวอักษรหรือเครื่องหมาย _ เท่านั้น ห้ามขึ้นต้นด้วยตัวเลข
- ห้ามมีช่องว่างหรืออักขระพิเศษ เช่น -, @, # (ยกเว้น _)
- คำนึงถึงตัวพิมพ์เล็ก-ใหญ่ (Case-Sensitive) เช่น Name และ name ถือเป็นตัวแปรคนละตัวกัน
- ห้ามใช้คำสงวน (Keyword) ของภาษา เช่น class, int, public เป็นชื่อตัวแปร
- นิยมตั้งชื่อแบบ **camelCase** สำหรับตัวแปรทั่วไป เช่น `studentName` และแบบ **PascalCase** สำหรับชื่อคลาสหรือเมธอด เช่น `ShowInfo`

2.3.5 ตัวอย่างการประกาศตัวแปรและแสดงผล

```
csharp
using System;

class Program
{
    static void Main(string[] args)
    {
        string studentName = "ปิยะดา ศรีสุข";
        int studentAge = 19;
        double gpa = 3.85;
        bool isGraduated = false;

        Console.WriteLine("ชื่อ: " + studentName);
        Console.WriteLine("อายุ: " + studentAge + " ปี");
        Console.WriteLine("เกรดเฉลี่ย: " + gpa);
        Console.WriteLine("จบการศึกษาแล้วหรือไม่: " + isGraduated);
    }
}
```

ผลลัพธ์เมื่อรันโปรแกรม

```
ชื่อ: ปิยะดา ศรีสุข
อายุ: 19 ปี
เกรดเฉลี่ย: 3.85
จบการศึกษาแล้วหรือไม่: False
```

2.3.6 ข้อผิดพลาดที่พบบ่อยเกี่ยวกับชนิดข้อมูล

ข้อผิดพลาด	สาเหตุ	ตัวอย่าง
แปลงชนิดข้อมูลไม่ถูกต้อง	นำ string ไปเก็บในตัวแปร int โดยตรง	int x = "18"; ❌
ตั้งชื่อตัวแปรผิดกฎ	ขึ้นต้นด้วยตัวเลข	int 1name = 5; ❌
แก้ไขค่าคงที่	พยายามกำหนดค่าใหม่ให้ const	PI = 3.14; ❌
ใช้ char ผิดเครื่องหมาย	ใช้เครื่องหมายคำพูดคู่แทนเดี่ยว	char g = "A"; ❌ (ต้องใช้ 'A')

2.4 การเขียนโปรแกรม C# เบื้องต้น: ประกาศคลาส สร้างวัตถุ และเรียกใช้เมธอด

2.4.1 ทบทวนแนวคิดจากหน่วยที่ 1

จากหน่วยที่ 1 ผู้เรียนได้ศึกษาแล้วว่า **คลาส (Class)** คือแม่แบบหรือพิมพ์เขียวที่กำหนดคุณลักษณะและพฤติกรรม เปรียบเหมือนแบบพิมพ์ขนม ส่วน **วัตถุ (Object)** คือสิ่งที่ถูกสร้างขึ้นจริงจากคลาสนั้น เปรียบเหมือนขนมที่กดออกมาจากแบบพิมพ์ หน่วยที่ 2 นี้จะแสดงให้เห็นว่าแนวคิดดังกล่าวถูกนำมาเขียนเป็นโค้ดจริงในภาษา C# ได้อย่างไร

2.4.2 ไวยากรณ์การประกาศคลาส

โครงสร้างพื้นฐานของการประกาศคลาสในภาษา C# ประกอบด้วย 2 ส่วนหลัก คือ **Attribute** (คุณลักษณะ/ข้อมูล) และ **Method** (พฤติกรรม/การทำงาน)

```

csharp

using System;

class Student // ประกาศคลาส Student
{
    // Attribute (คุณลักษณะ)
    public string Name;
    public string StudentId;
    public double Gpa;

    // Method (พฤติกรรม)
    public void ShowInfo()
    {
        Console.WriteLine("รหัสนักศึกษา: " + StudentId);
        Console.WriteLine("ชื่อ: " + Name);
        Console.WriteLine("เกรดเฉลี่ย: " + Gpa);
    }
}

class Program
{
    static void Main(string[] args)
    {
        // สร้างวัตถุ (Object) จากคลาส Student ด้วยคำสั่ง new
        Student somchai = new Student();
        somchai.Name = "สมชาย ใจดี";
        somchai.StudentId = "6501001";
        somchai.Gpa = 3.60;

        somchai.ShowInfo(); // เรียกใช้เมธอดของวัตถุ
    }
}

```

ผลลัพธ์เมื่อรันโปรแกรม

```

รหัสนักศึกษา: 6501001
ชื่อ: สมชาย ใจดี
เกรดเฉลี่ย: 3.6

```

2.4.3 คำอธิบายทีละขั้นตอน

โค้ด	ความหมาย
class Student { ... }	สร้างแม่แบบชื่อ Student กำหนดคุณลักษณะ 3 อย่าง และพฤติกรรม 1 อย่าง
Student somchai = new Student();	สร้างวัตถุ (Instance) จากคลาส Student คำสั่ง new จัดสรรหน่วยความจำใหม่
somchai.Name = "สมชาย ใจดี";	กำหนดค่าให้คุณลักษณะของวัตถุผ่านเครื่องหมายจุด (Dot Operator)
somchai.ShowInfo();	เรียกใช้เมธอดของวัตถุ ทำให้วัตถุแสดงพฤติกรรมตามที่กำหนด

ตัวแปร somchai ทำหน้าที่เป็นตัวอ้างอิง (Reference) ไปยังวัตถุที่ถูกสร้างขึ้นในหน่วยความจำ ไม่ใช่ตัววัตถุโดยตรง

2.4.4 ตัวอย่างการสร้างวัตถุหลายตัวจากคลาสเดียวกัน

```
using System;

class Student
{
    public string Name;

    public string StudentId;

    public double Gpa;

    public void ShowInfo()
    {
        Console.WriteLine("รหัสนักศึกษา: " + StudentId + " | ชื่อ: " + Name + " | GPA: " + Gpa);
    }
}

class Program
{
    static void Main(string[] args)
    {
        Student student1 = new Student();
```

```

student1.Name = "สมชาย ใจดี";

student1.StudentId = "6501001";

student1.Gpa = 3.60;

Student student2 = new Student();

student2.Name = "สมหญิง รักเรียน";

student2.StudentId = "6501002";

student2.Gpa = 3.90;

student1.ShowInfo();

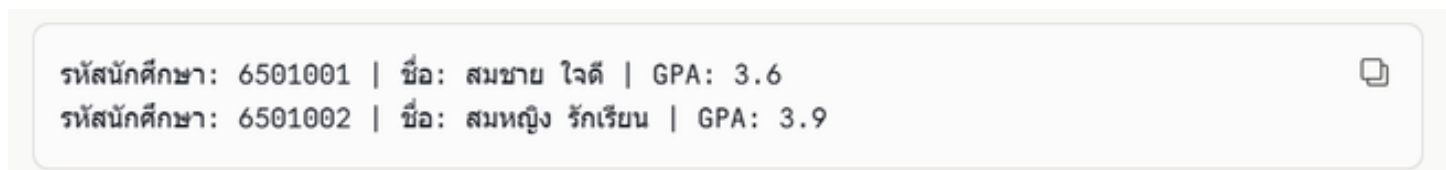
student2.ShowInfo();

}

}

```

ผลลัพธ์เมื่อรันโปรแกรม



จะเห็นได้ว่า student1 และ student2 เป็นวัตถุคนละตัวกัน แม้จะถูกสร้างจากคลาส Student เดียวกัน แต่ค่าของคุณลักษณะภายในเป็นอิสระต่อกันโดยสิ้นเชิง

2.4.5 ตัวอย่างเพิ่มเติม: คลาส "สินค้า" (Product)

เพื่อให้เห็นภาพการประยุกต์ใช้ในบริบทอื่น ขอยกตัวอย่างคลาสที่เกี่ยวข้องกับร้านค้า

```

using System;

class Product

{

    public string ProductName;

    public double Price;

    public int Stock;

    public void ShowProduct()

    {

        Console.WriteLine("สินค้า: " + ProductName);
    }
}

```

```

        Console.WriteLine("ราคา: " + Price + " บาท");

        Console.WriteLine("คงเหลือ: " + Stock + " ชิ้น");

    }

    public void Sell(int quantity)

    {

        if (quantity <= Stock)

        {

            Stock -= quantity;

            Console.WriteLine("ขายสำเร็จ " + quantity + " ชิ้น");

        }

        else

        {

            Console.WriteLine("สินค้าไม่พอขาย");

        }

    }

}

class Program

{

    static void Main(string[] args)

    {

        Product notebook = new Product();

        notebook.ProductName = "สมุดโน้ต";

        notebook.Price = 25.0;

        notebook.Stock = 100;

        notebook.ShowProduct();

        notebook.Sell(10);

        notebook.ShowProduct();

    }

}

```

สินค้า: สมุดโน้ต
ราคา: 25 บาท
คงเหลือ: 100 ชิ้น
ขายสำเร็จ 10 ชิ้น
สินค้า: สมุดโน้ต
ราคา: 25 บาท
คงเหลือ: 90 ชิ้น

ตัวอย่างนี้แสดงให้เห็นว่าเมธอด `Sell()` สามารถเปลี่ยนแปลงค่าของคุณลักษณะ `Stock` ภายในวัตถุได้ตามเงื่อนไขที่กำหนด ซึ่งเป็นตัวอย่างพื้นฐานของการที่วัตถุมีพฤติกรรมที่ส่งผลต่อสถานะข้อมูลของตนเอง

2.4.6 ข้อผิดพลาดที่พบบ่อยเมื่อเริ่มเขียนคลาสและวัตถุ

ข้อผิดพลาด	สาเหตุ	วิธีแก้ไข
ลืมใช้คำสั่ง <code>new</code>	เขียน <code>Student s</code> ; แล้วเรียกใช้ทันที	ต้องเขียน <code>Student s = new Student();</code> ก่อนเสมอ
เรียกเมธอดผิดชื่อ (Case-Sensitive)	เขียน <code>showinfo()</code> แทน <code>ShowInfo()</code>	ตรวจสอบตัวพิมพ์เล็ก-ใหญ่ให้ตรงกับที่ประกาศ
ลืมเครื่องหมายจุด	เขียน <code>somchai Name = ...</code>	ต้องใช้ <code>somchai.Name = ...</code>
Access Modifier ไม่ถูกต้อง	ประกาศ Attribute เป็น <code>private</code> แล้วเข้าถึงจากภายนอก	ใช้ <code>public</code> หากต้องการเข้าถึงจากภายนอกคลาส

2.5 การประยุกต์คุณสมบัติ OOP ด้วยภาษา C#

2.5.1 การห่อหุ้ม (Encapsulation)

หลักการ: การห่อหุ้มคือการรวมข้อมูล (Attribute) และพฤติกรรม (Method) ไว้เป็นหน่วยเดียวกันภายในวัตถุ พร้อมทั้งซ่อนรายละเอียดการทำงานภายในไม่ให้ภายนอกเข้าถึงได้โดยตรง เปรียบเหมือนตู้ ATM ที่ผู้ใช้เห็นเพียงปุ่มกด แต่ไม่สามารถเข้าถึงกลไกภายในได้โดยตรง ในภาษา C# ใช้ตัวระบุระดับการเข้าถึง (**Access Modifier**) เพื่อควบคุมการเข้าถึงข้อมูล

ตัวระบุ	ความหมาย
public	เข้าถึงได้จากทุกส่วนของโปรแกรม
private	เข้าถึงได้เฉพาะภายในคลาสเดียวกันเท่านั้น
protected	เข้าถึงได้ภายในคลาสเดียวกันและคลาสที่สืบทอดต่อ

และนิยมใช้ **Property** ซึ่งเป็นกลไกพิเศษของภาษา C# เพื่อควบคุมการอ่าน (get) และเขียน (set) ค่าของข้อมูลอย่างปลอดภัย

using System;

class BankAccount

```
{
    private double balance;    // ข้อมูลถูกซ่อนไว้ภายใน (private)
    public double Balance      // Property สำหรับเข้าถึงข้อมูลอย่างปลอดภัย
    {
        get { return balance; }
        private set { balance = value; }
    }
    public void Deposit(double amount)
    {
        if (amount > 0)
        {
            balance += amount;
            Console.WriteLine("ฝากเงินสำเร็จ: " + amount + " บาท");
        }
        else
        {
            Console.WriteLine("จำนวนเงินไม่ถูกต้อง");
        }
    }
}
```



```

    }

}

class Program
{
    static void Main(string[] args)
    {
        BankAccount account = new BankAccount();

        account.Deposit(1000);

        account.Deposit(-500);    // ถูกป้องกันโดยเงื่อนไขในเมธอด

        Console.WriteLine("ยอดคงเหลือ: " + account.Balance + " บาท");
    }
}

```

ผลลัพธ์เมื่อรันโปรแกรม

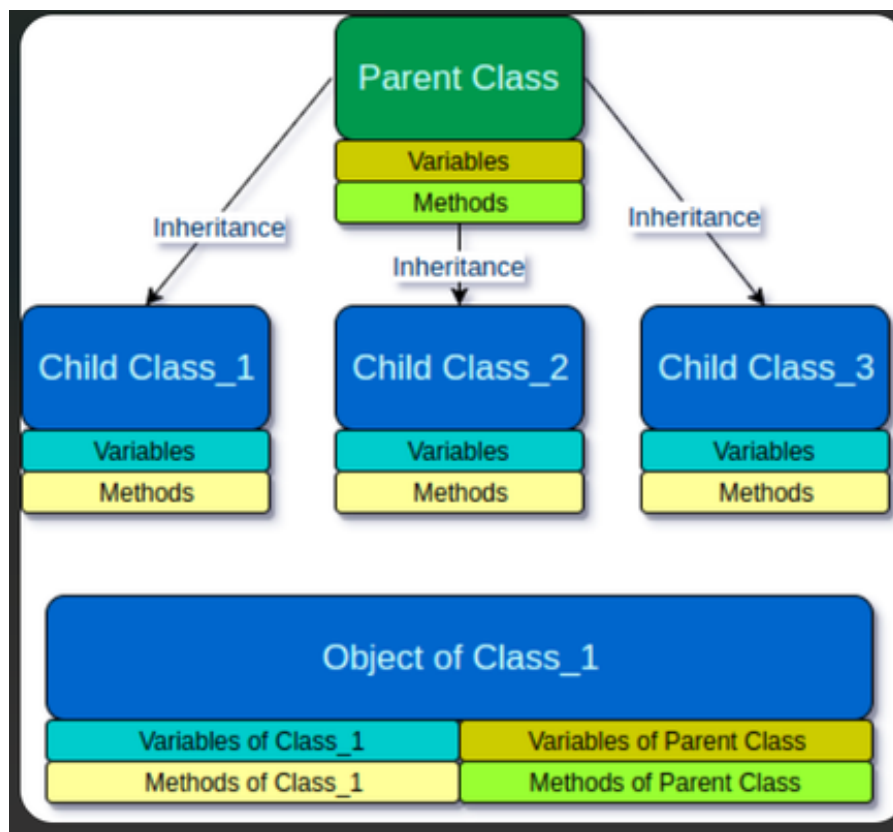
```

ฝากเงินสำเร็จ: 1000 บาท
จำนวนเงินไม่ถูกต้อง
ยอดคงเหลือ: 1000 บาท

```

คำอธิบาย: ตัวแปร balance ถูกกำหนดเป็น private ทำให้ภายนอกคลาสไม่สามารถเข้าถึงหรือแก้ไขค่าได้โดยตรง ผู้ใช้จำเป็นต้องเรียกผ่านเมธอด Deposit() หรืออ่านค่าผ่าน Property Balance เท่านั้น ซึ่งช่วยป้องกันการกำหนดค่าที่ไม่ถูกต้อง เช่น การฝากเงินจำนวนติดลบ

2.5.2 การสืบทอด (Inheritance)



ภาพที่ 6 การสืบทอด

ที่มา <https://www.tutorialkart.com/java/inheritance-in-java/>

หลักการ: การสืบทอดคือการที่คลาสใหม่ (คลาสลูก) สามารถรับเอาคุณลักษณะและเมธอดของคลาสเดิม (คลาสแม่) มาใช้งานได้ทันที โดยไม่ต้องเขียนใหม่ทั้งหมด และยังสามารถเพิ่มเติมความสามารถเฉพาะของตัวเองได้ เปรียบเหมือนลูกที่ได้รับลักษณะทางพันธุกรรมจากพ่อแม่ และมีลักษณะเฉพาะของตนเองเพิ่มเติม

ในภาษา C# ใช้เครื่องหมาย : (colon) ตามหลังชื่อคลาสลูก เพื่อระบุว่าสืบทอดจากคลาสใด

```
using System;

class Vehicle           // คลาสแม่
{
    public string Brand;

    public void Move()
    {
        Console.WriteLine("ยานพาหนะกำลังเคลื่อนที่");
    }
}

class Car : Vehicle     // คลาสลูกสืบทอดจาก Vehicle
{

```

```

public int Doors;

public void Horn()

{
    Console.WriteLine("บีบ ๆ");
}

}

class Program
{
    static void Main(string[] args)
    {
        Car myCar = new Car();

        myCar.Brand = "Toyota";    // สืบทอดมาจาก Vehicle

        myCar.Doors = 4;           // เฉพาะของ Car

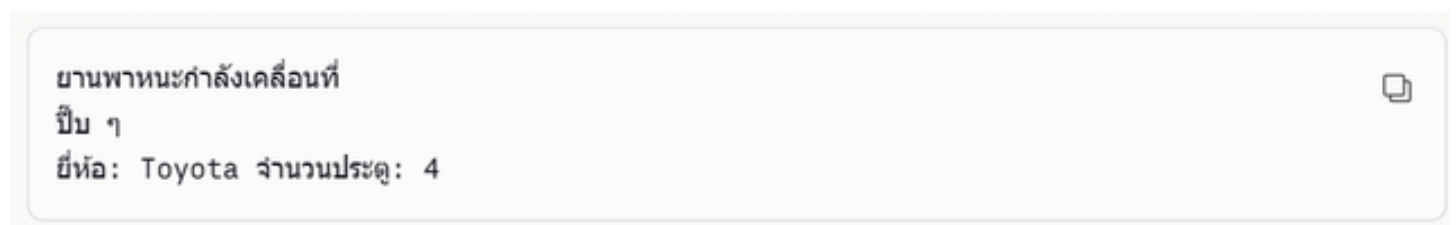
        myCar.Move();              // สืบทอดมาจาก Vehicle

        myCar.Horn();              // เมธอดเฉพาะของ Car

        Console.WriteLine("ยี่ห้อ: " + myCar.Brand + " จำนวนประตู: " + myCar.Doors);
    }
}

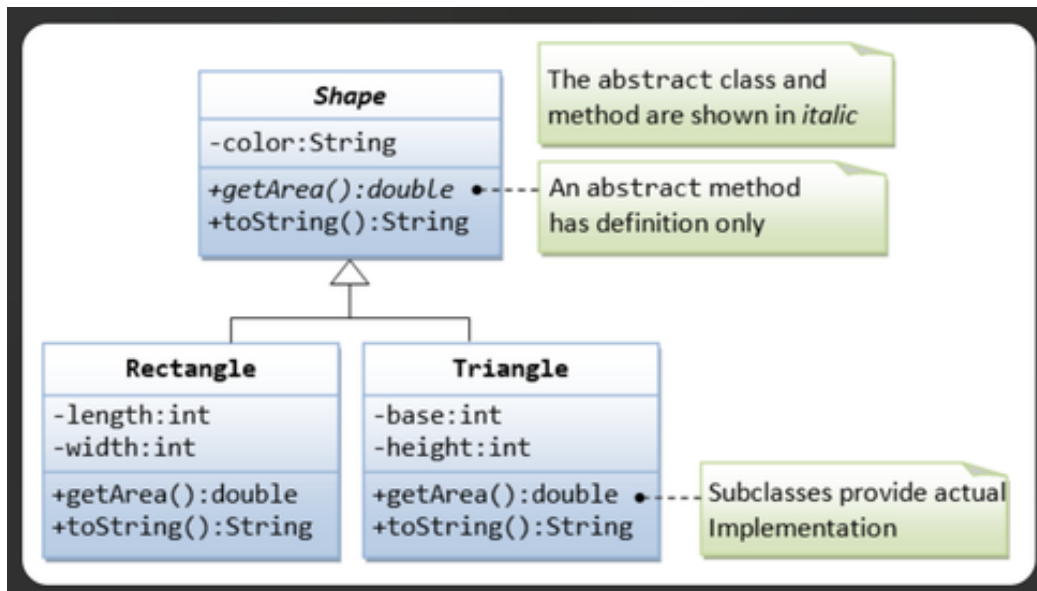
```

ผลลัพธ์เมื่อรันโปรแกรม



คำอธิบาย: เมื่อสร้างวัตถุจากคลาส Car วัตถุนั้นจะมีทั้งคุณลักษณะ Brand และเมธอด Move() ที่สืบทอดมาจากคลาส Vehicle และยังมีคุณลักษณะ Doors กับเมธอด Horn() ที่เพิ่มขึ้นมาเฉพาะของคลาส Car เอง

2.5.3 การพ้องรูป (Polymorphism)



ภาพที่ 7 การพ้องรูป

ที่มา <https://fity.club/lists/suggestions/Java-Inheritance-Tutorial/>

หลักการ: การพ้องรูปคือการที่เมธอดชื่อเดียวกันสามารถทำงานแตกต่างกันไปตามชนิดของวัตถุที่เรียกใช้ ในภาษา C# ทำได้โดยใช้คำสั่ง `virtual` ในคลาสแม่เพื่อประกาศว่าเมธอดนี้สามารถถูกเปลี่ยนแปลงพฤติกรรมได้ และใช้ `override` ในคลาสลูกเพื่อเขียนพฤติกรรมใหม่ที่ทับของเดิม

```
using System;
```

```
class Shape
```

```
{
```

```
    public virtual void Draw()
```

```
    {
```

```
        Console.WriteLine("วาดรูปทรงทั่วไป");
```

```
    }
```

```
}
```

```
class Circle : Shape
```

```
{
```

```
    public override void Draw()
```

```
    {
```

```
        Console.WriteLine("วาดวงกลม");
```

```
    }
```

```

}

class Square : Shape

{

    public override void Draw()

    {

        Console.WriteLine("วาดสี่เหลี่ยม");

    }

}

class Program

{

    static void Main(string[] args)

    {

        Shape[] shapes = new Shape[2];

        shapes[0] = new Circle();

        shapes[1] = new Square();

        foreach (Shape s in shapes)

        {

            s.Draw();    // คำสั่งเดียวกัน แต่ผลลัพธ์ต่างกันตามชนิดวัตถุจริง

        }

    }

}

```

ผลลัพธ์เมื่อรันโปรแกรม

วาดวงกลม
วาดสี่เหลี่ยม



คำอธิบาย: แม้จะเรียกใช้เมธอด Draw() ด้วยคำสั่งเดียวกันในรูป foreach แต่ผลลัพธ์ที่ได้แตกต่างกันไปตามชนิดของวัตถุจริงที่ถูกอ้างอิงอยู่ในขณะนั้น นี่คือแก่นสำคัญของการพ้องรูป

2.5.4 การเป็นนามธรรม (Abstraction)

หลักการ: การเป็นนามธรรมคือการแสดงเฉพาะสิ่งที่จำเป็นต่อการใช้งาน และซ่อนรายละเอียดความซับซ้อนภายในเอาไว้ ในภาษา C# ทำได้โดยใช้คำสั่ง abstract เพื่อกำหนดคลาสต้นแบบที่ยังไม่สมบูรณ์ในตัวเอง ไม่สามารถสร้างวัตถุได้โดยตรง และบังคับให้คลาสลูกต้องนำไปพัฒนาต่อให้สมบูรณ์

```
using System;
```

```
abstract class Animal
```

```
{  
  
    public abstract void Speak();    // เมธอดนามธรรม ไม่มีเนื้อหาการทำงาน  
  
}
```

```
class Dog : Animal
```

```
{  
  
    public override void Speak()  
  
    {  
  
        Console.WriteLine("โห้ง ๆ");  
  
    }  
  
}
```

```
class Cat : Animal
```

```
{  
  
    public override void Speak()  
  
    {  
  
        Console.WriteLine("เหมียว");  
  
    }  
  
}
```

```
class Program
```

```
{  
  
    static void Main(string[] args)  
  
    {  
  
        Animal myDog = new Dog();  
  
        Animal myCat = new Cat();  
  
        myDog.Speak();
```



```
myCat.Speak();
```

```
// Animal a = new Animal(); // ❌ ทำไม่ได้ เพราะเป็น abstract class
```

```
}
```

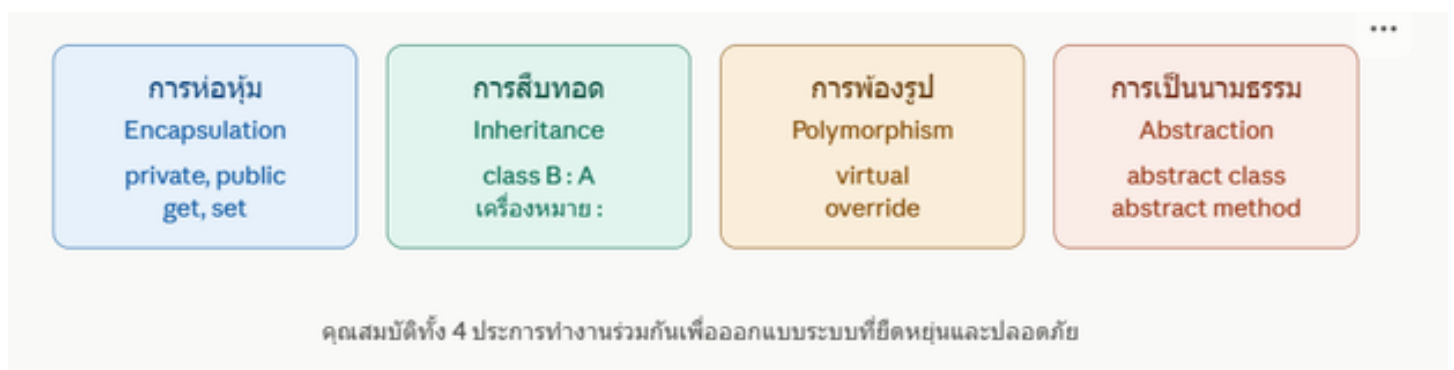
```
}
```

ผลลัพธ์เมื่อรันโปรแกรม

โง่ง ๆ
เหมียว

คำอธิบาย: คำสั่ง abstract class Animal ระบุว่าคลาส Animal เป็นเพียงต้นแบบ ไม่สามารถเขียน new Animal() เพื่อสร้างวัตถุได้โดยตรง เมธอด Speak() ที่ประกาศเป็น abstract ไม่มีเนื้อหาการทำงานใด ๆ อยู่ในคลาสแม่ แต่บังคับให้ทุกคลาสลูกที่สืบทอดไปต้องเขียนเนื้อหาการทำงานของเมธอดนี้ขึ้นมาเอง

ต่อไปนี้เป็นแผนภาพสรุปคุณสมบัติ OOP ทั้ง 4 ประการในภาษา C# พร้อมคำสั่งหลักที่ใช้



ภาพที่ 8 สรุปคุณสมบัติ OOP ทั้ง 4 ประการ

สรุปท้ายบทเรียน

2.1 ความเป็นมา ความสำคัญ และคุณลักษณะเด่น

ภาษา C# พัฒนาโดยไมโครซอฟท์ตั้งแต่ปี ค.ศ. 2000 โดยมี **Anders Hejlsberg** เป็นหัวหน้าทีมออกแบบ ทำงานบนแพลตฟอร์ม .NET ผ่านกลไก **CLR (Common Language Runtime)** มีคุณลักษณะเด่นคือเป็นภาษาเชิงวัตถุแท้ กำหนดชนิดข้อมูลอย่างเข้มงวด (Type-Safe) จัดการหน่วยความจำอัตโนมัติผ่าน Garbage Collector และเป็นโอเพนซอร์สที่ใช้งานข้ามแพลตฟอร์มได้ นิยมใช้พัฒนาโปรแกรมองค์กร เว็บแอปพลิเคชัน (ASP.NET Core) และเกม (Unity)

2.2 องค์ประกอบโครงสร้างโปรแกรมและสภาพแวดล้อมการพัฒนา

การพัฒนาโปรแกรม C# ต้องอาศัย **.NET SDK** เป็นแกนหลัก (Compiler, Runtime, Library, CLI) ร่วมกับเครื่องมือแก้ไขโค้ด โดยมีให้เลือกระหว่าง **Visual Studio** (IDE ครบวงจร เหมาะกับผู้เริ่มต้น) และ **Visual Studio Code** (Editor เบา ยืดหยุ่นสูง ต้องติดตั้ง Extension เพิ่ม)

2.3 การจำแนกชนิดข้อมูลและตัวแปร

C# เป็นภาษา **Strongly Typed** ตัวแปรทุกตัวต้องระบุชนิดข้อมูลชัดเจน แบ่งเป็น 3 กลุ่มหลัก คือ ตัวเลข (int, double, decimal), ข้อความ (string, char) และตรรกะ (bool) พร้อมมีกฎการตั้งชื่อตัวแปรและการประกาศค่าคงที่ด้วย const

2.4 การประกาศคลาส สร้างวัตถุ และเรียกใช้เมธอด

คลาส คือแม่แบบที่กำหนด Attribute และ Method ส่วน **วัตถุ** คือสิ่งที่สร้างขึ้นจริงด้วยคำสั่ง new คลาสเดียวสามารถสร้างวัตถุได้หลายตัวโดยแต่ละตัวมีข้อมูลเป็นอิสระต่อกัน เข้าถึงคุณลักษณะและเรียกใช้เมธอดผ่านเครื่องหมายจุด (Dot Operator)

2.5 การประยุกต์คุณสมบัติ OOP ทั้ง 4 ประการ

คุณสมบัติ	คำสั่งหลัก	แก่นสำคัญ
Encapsulation	private, public, Property	ซ่อนข้อมูล ป้องกันการเข้าถึงโดยตรง
Inheritance	: (colon)	คลาสลูกรับคุณสมบัติจากคลาสแม่
Polymorphism	virtual / override	เมธอดชื่อเดียวกัน ทำงานต่างกันตามวัตถุ
Abstraction	abstract class	แสดงเฉพาะสิ่งจำเป็น ซ่อนรายละเอียดซับซ้อน